EE5203 L13 Study Sheet

Demonstration, oral defence, and reflection - Basri Erdogan

Essential question: The system either works or it does not — that is settled by now. The remaining question is whether you can explain it: point at any wire, any register, any line, and say why it is that way and not another.

What you should be able to do

- Run a rehearsed demonstration with a working fallback.
- State each requirement with its test and the result you observed.
- Inject a fault deliberately and explain the system's response.
- Explain your own contribution at register level.
- Justify a design decision against the alternative you rejected.
- Diagnose a live failure out loud, using your own instruments.

What is assessed

Component	Weight	Team / individual
Demonstration against requirements	35 %	team
Report	25 %	team
Presentation	15 %	team
Individual oral defence	**25 %**	**individual**

Two members of one team routinely receive different defence marks. That is the purpose of having one.

The demonstration

Time	Segment
0-2	Setup: it runs before you start explaining
2-7	Requirements walk: one requirement, one test, one result
7-12	Live operation, including a deliberately injected fault
12-15	Known issues: specific and honest

Run first, talk over it. A demo that opens with five minutes of slides has spent its best minutes on the wrong thing.

The injected fault

A system that only works is a demo. A system whose failure you can predict, trigger, and explain is engineering — and almost nobody does it.

Options, all rehearsable:

- disconnect a sensor mid-run;
- hold a breakpoint until `ORE` sets, then show recovery or its absence;
- send an out-of-range command and show it refused;
- stall an actuator and show the current draw or the reset.

State the prediction before you trigger it. Predicting correctly is worth more than the failure being graceful.

When it fails for real

1. **Say what should have happened.** You know the expected behaviour.
2. **Name the species** — component, resource, or timing (L12).
3. **Use your instruments** — telemetry, counters, the SFR view.
4. **Fall back** to the tagged build.

Do not start editing code. A convincing live diagnosis can outscore a demo that worked but cannot be explained.

The report: decisions, not facts

A fact (free)	A decision (a mark)
"We used TIM3 for the servo."	"We put the servo on TIM3 rather than TIM2 because TIM2 already ran the LED at 1 kHz, and channels of one timer share `ARR`. The servo needs 50 Hz, so it needed its own timer."
"We send data over UART."	"We send CSV at 10 Hz rather than `printf` per character, because one blocking transmit of 18 characters costs 1.56 ms and per-character transmits would cost 18 times that."

Contents: requirements and results · design decisions and rejected alternatives · resource map and timing budget · evidence (plots, measurements, register screenshots) · known issues and future work.

The individual defence

Three levels of question:

1. **Your contribution** — "Which parts did you write? Walk me through one."
2. **The mechanism** — "Which register does that configure? What does the hardware do with it?"
3. **The alternative** — "Why this way? What would happen if you did it the other way?"

Level 3 is where marks are won. Anyone can describe what they did; explaining why the alternative is worse shows you understood the choice.

The six questions to prepare for

- Which parts did you write, and which resources do they own?
- Show me where you configure that peripheral. Why those values?
- What is in that register right now, and why?
- What would happen if I removed this line?
- How do you know this requirement is met? Show me the evidence.
- What broke during the term, and how did you find it?

None can be scripted. All can be prepared by having done the work.

"I only did the software"

Division of labour is expected. Not knowing where your own code's signals go is not. Be able to say:

- which pins your code touches, and where they go physically;
- what your part costs in the timing budget;
- which requirement your part is responsible for.

Reflection

1. What would you design differently, knowing what you know now?
2. What cost you the most time, and what would have prevented it?

"Nothing, it went well" scores nothing. The specific answers are the most valuable engineering output of the term.

Common mistakes

- Unrehearsed: discovering the projector, the power, or the order live.
- No tagged fallback.
- One person knows everything — fine for the demo, expensive at the defence.
- Hiding known issues; they are found anyway, at higher cost.
- Claiming a requirement is met without the measurement.
- Reading the slides. You built it; talk about it.
- Changing code the morning of the demonstration.

Mastery self-check

- ☐ I have rehearsed the full 15 minutes on the hardware.
- ☐ I have a tagged build I can flash in under two minutes.
- ☐ I can state every requirement with its test and result.
- ☐ I have chosen and rehearsed a fault to inject, with a prediction.
- ☐ I can explain my own contribution at register level.
- ☐ I can justify one design decision against its alternative.
- ☐ I have written both reflection answers, specifically.

Five review questions

1. Turn "we used interrupts for the button" into a design decision, with the rejected alternative and the reason.
2. Your demo fails in the first minute. Write the four sentences you say next, in order.
3. You are asked "what is in `TIM2->CCR1` right now, and why?" Describe how you would answer for your own project.
4. Name a fault you could inject in your system, the prediction you would state first, and the evidence you would show.
5. Answer both reflection questions for your own project, in three sentences each. Be specific enough that another team could act on the advice.